VISUAL CALCULUS

for Machine Learning and Artificial Intelligence

Understanding what the maths actually *means* — not how to pass an exam.

Inspired by the visual teaching philosophy of Grant Sanderson (3Blue1Brown)

Every concept explained through geometry, intuition and real ML applications. Formulas appear only as footnotes — what matters is the picture in your head.

CHAPTERS

1. **What is a Function?** — The building block of everything
2. **Limits — The Language of Infinity** — What does 'approaching' mean?
3. **The Derivative** — Instantaneous rate of change
4. **Rules of Differentiation** — Chain, product, and power rules visually
5. **The Integral** — Accumulation and area
6. **Fundamental Theorem of Calculus** — How integration and differentiation link
7. **Multivariable Calculus** — Functions of many inputs (weight spaces)
8. **Gradient Descent** — ★ The engine of machine learning
9. **Backpropagation** — ★ Chain rule inside neural networks
10. **Taylor Series** — Approximating any function with polynomials
11. **Differential Equations** — How quantities evolve over time
12. **Jacobian and Hessian** — ★ Advanced optimization geometry
13. **Calculus and Probability** — ★ Distributions, KL divergence, entropy
14. **Attention and Softmax** — ★ Calculus inside Transformers

★ = especially relevant to ML / AI

Chapter 1

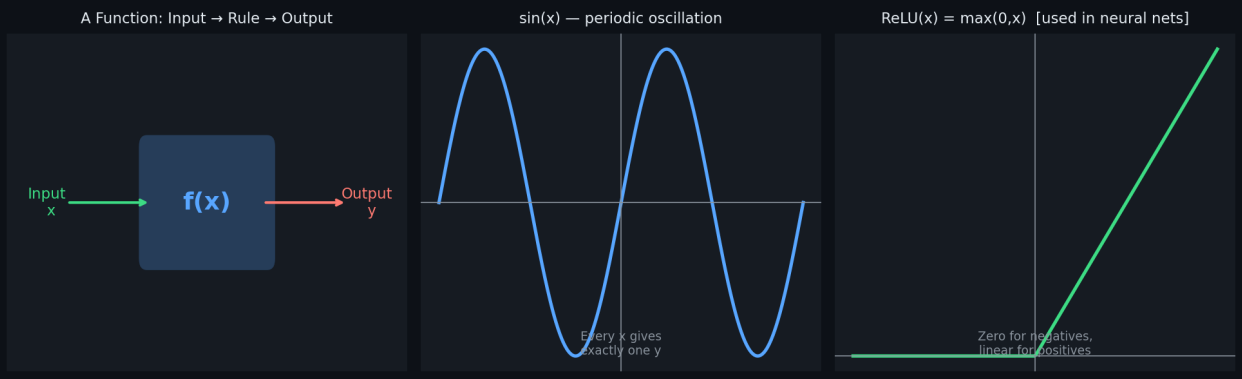
What is a Function?

Before calculus makes any sense, you need to be friends with functions. Not the formula — the *idea*. A function is a machine: put something in, get something out. Every output must be uniquely determined by its input. That is the only rule.

The Machine Metaphor

Imagine a vending machine. Press B3, you get exactly one snack — always the same one. That is a function. The button is your input x. The snack is your output f(x). What makes it a function is the guarantee: same button, same snack, every time. No randomness. No ambiguity.

Functions do not have to be smooth curves. They can have corners, jumps, or holes. What they cannot have is one input giving two different outputs simultaneously. That breaks the machine.



Left: the machine metaphor. Middle: sin(x) — a smooth periodic function. Right: ReLU(x) — the most common activation function in modern neural networks.

Why Functions Matter in ML

A neural network is, at its heart, a very large composite function. You feed it an image (the input) and it produces a label, a probability, or a sentence (the output). Every layer is a function. The whole network is functions nested inside functions inside functions. Understanding this means understanding that *training a network* is just finding the best version of that composite function.

> **ML / AI Connection**
> Loss function, activation function, transformation function — all are the same concept. A neural network with n layers computes: output = f_n(f_{n-1}(...f_1(x)...)). Each f_i is a function. Calculus is the tool we use to adjust each f_i to reduce error.

Footnote: formally, f: X → Y where for every x in X there exists exactly one y in Y such that f(x) = y. Common notation: y = f(x), output = f(input).

Chapter 2

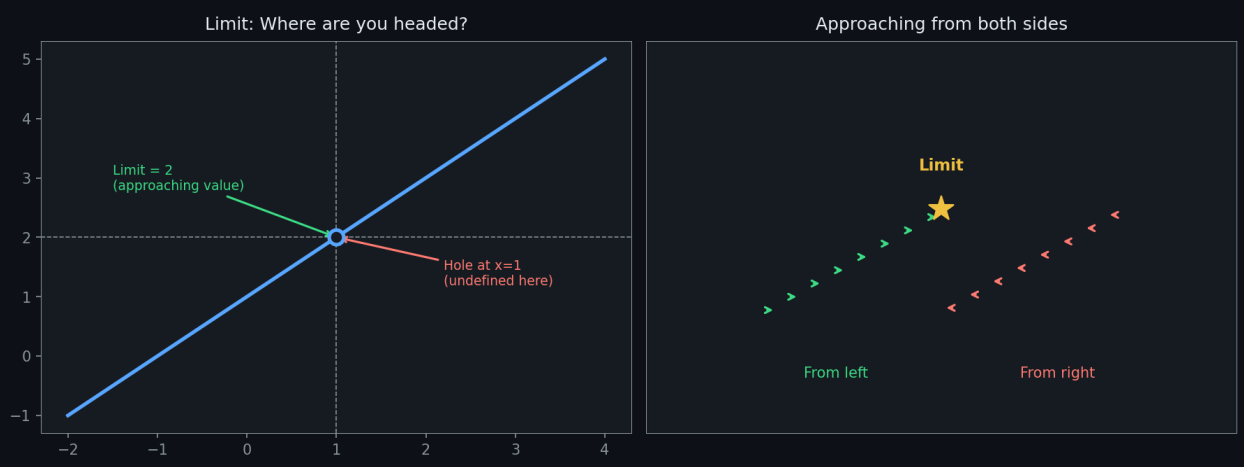
Limits — The Language of Infinity

A limit asks a beautiful question: if I keep walking toward a destination, where am I headed — even if I never actually arrive? This idea is the foundation stone of all of calculus. Derivatives and integrals are both defined using limits.

The Intuition: Where Are You Going?

Picture a road that leads to a town you are not allowed to enter. You can walk arbitrarily close. You can see the town clearly. The limit is the town itself — your destination — even if you never step inside. The function's value at the exact point might not exist (a hole), but the limit — the approach — is perfectly well defined.

This is the core insight: a limit is about *behavior near a point*, not *behavior at a point*. These can be different things.



Left: a function with a hole at x=1. The function is undefined there, but the limit is 2 — the value being approached. Right: the same limit seen as arrows converging from both sides.

Left and Right Limits

You can approach a point from the left side or the right side. If both sides agree on where you are heading, the limit exists. If they disagree — like a cliff edge where left and right give different heights — the overall limit does not exist. This is not a failure; it is a discovery about the function's shape.

In machine learning, we encounter a version of this when we look at whether a loss surface has sharp corners. A corner means the left and right limits of the slope disagree — which has direct consequences for gradient-based training.

Continuity: No Surprises

A function is continuous at a point if the limit equals the actual value. No holes, no jumps, no teleportation. Most functions you will encounter in ML are continuous almost everywhere. The

exceptions — like ReLU at x=0 — are carefully managed.

ML / AI Connection

ReLU(x) = max(0, x) has a kink at x=0. The limit from the left is 0, the limit from the right is 0. They agree, so the limit exists. But the derivative does not — the slope jumps. This is why subgradients exist.

Footnote: lim_{x→a} f(x) = L means that for any epsilon > 0, there exists delta > 0 such that |x - a| < delta implies |f(x) - L| < epsilon.

Chapter 3

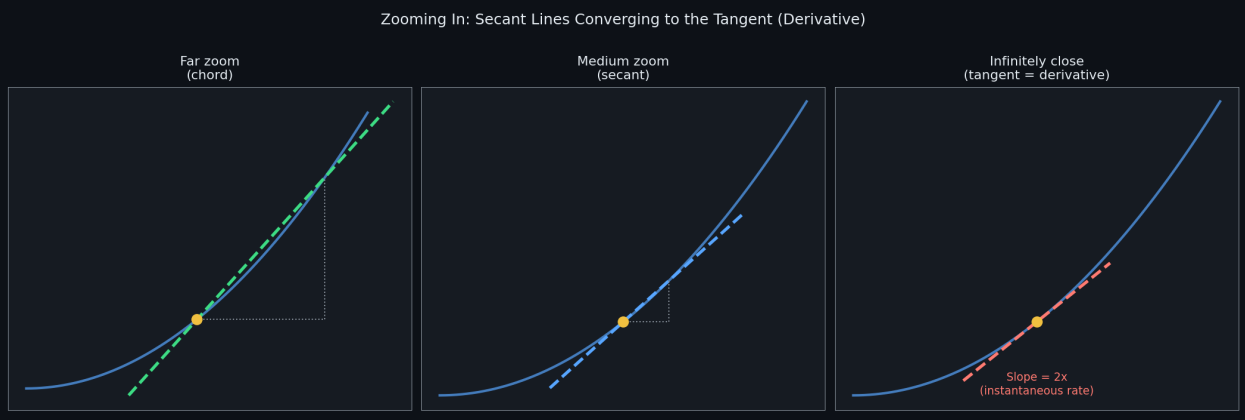
The Derivative

The derivative is simply this: the *instantaneous rate of change*. Not the average over a period. Not over a minute or a second. At exactly this moment, how fast is the thing changing? Visually, it is the slope of the curve at a single point.

Zooming In Until You See a Straight Line

Take any smooth curve. Pick a point on it. Zoom in. Keep zooming. At some level of magnification, every smooth curve looks like a straight line. The slope of that line is the derivative at that point.

This is the visual truth that all the formulas are describing. The derivative is not a ratio of infinitely small things — it is the slope you see when you zoom infinitely close.

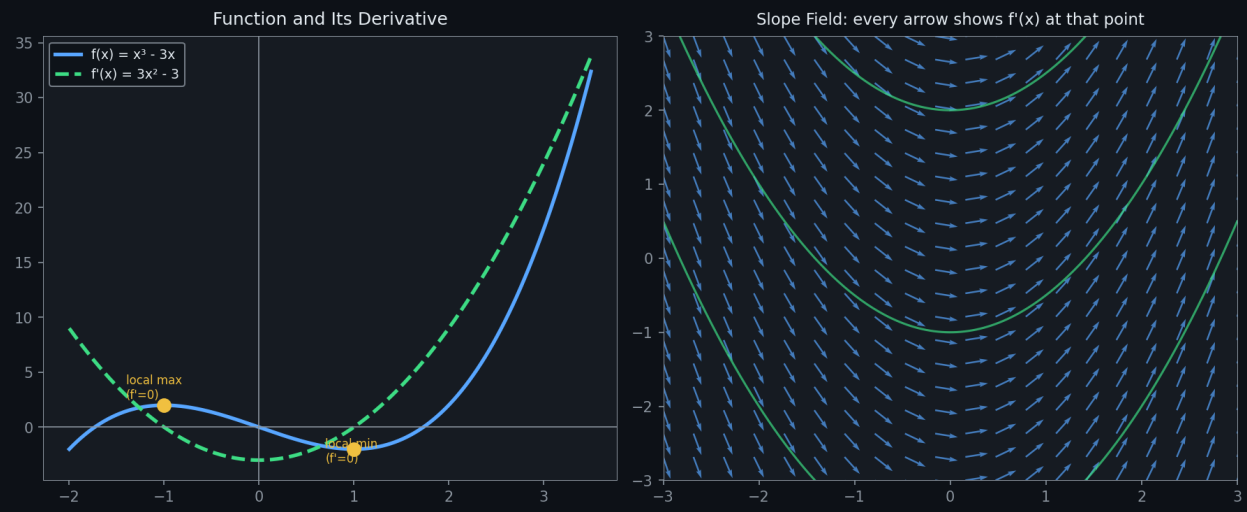


Zooming in on $f(x) = x^2$ at $x=2$. A chord between two points (left) becomes the tangent line (right) as we bring the two points infinitely close. That tangent slope is the derivative: $f'(2) = 4$.

The Derivative as a New Function

The derivative is not just a single number. As you slide your point across the curve, the slope changes — it is itself a function. If f(x) is a function, f'(x) (read: 'f prime of x') is the derivative function that tells you the slope at every point.

Where f'(x) = 0: the slope is flat — a peak, a valley, or a plateau. Where f'(x) > 0: the function is climbing. Where f'(x) < 0: the function is descending. These three facts drive almost all of optimization.



Left: f(x) = x³ - 3x and its derivative f'(x) = 3x² - 3. The yellow dots mark where the derivative is zero — peaks and valleys. Right: a slope field where every arrow shows the derivative at that location.

ML / AI Connection

In a neural network with loss L and weight w, the derivative dL/dw tells us: 'if I nudge weight w up by a tiny amount, does the loss go up or down, and by how much?' This is the fundamental signal used to train every neural network ever built.

Footnote: f'(x) = lim_{h→0} [f(x+h) - f(x)] / h. Common notation: dy/dx, Df, f'(x), or df/dx.

Chapter 4

Rules of Differentiation

The derivative's definition involves a limit calculation every time — which would be tedious. Mathematicians discovered patterns: shortcut rules that let you find derivatives by recognition rather than calculation. The one that matters most for ML is the Chain Rule.

The Power Rule — Slopes of Polynomial Curves

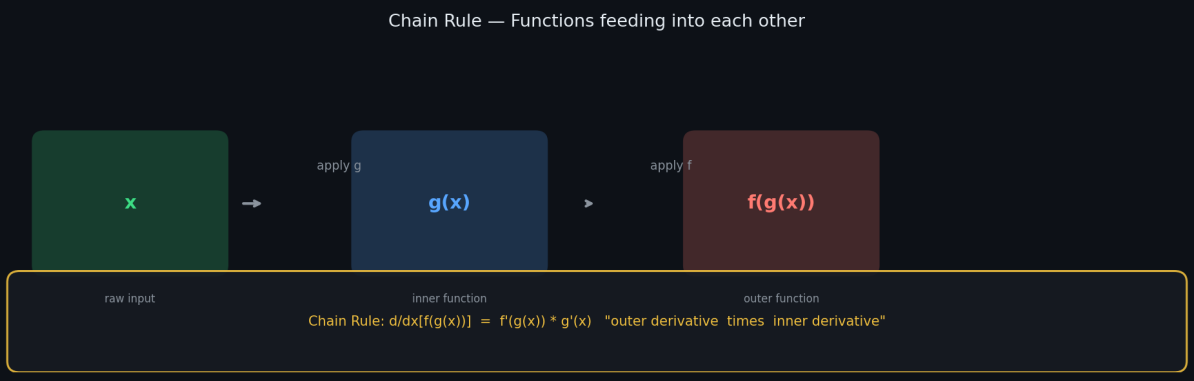
For any power of x, the derivative follows a visual pattern: bring the exponent down, reduce it by one. This is not magic — it comes from the zooming-in argument. A curve that grows as x-squared has a slope that grows linearly. A cubic curve has a quadratic slope. Each derivative is one step simpler.

Power rule (footnote): d/dx[x^n] = n * x^(n-1)

The Chain Rule — The Most Important Rule in ML

The chain rule handles *composed functions* — functions inside functions. The visual intuition: if spinning a gear A rotates gear B by 3x, and gear B rotates gear C by 2x, then A rotating makes C spin 6x. Rates of change multiply through the chain.

When you change x slightly, it changes g(x) a bit. That change in g flows through to change f(g(x)). The total effect is the product of both individual rates of change.



The chain rule as a pipeline: each stage converts an input nudge into an output nudge. The overall sensitivity is the product of all individual sensitivities.

ML / AI Connection

Backpropagation IS the chain rule — applied layer by layer through a neural network. When computing how the loss depends on a weight deep in the network, you multiply derivatives through every layer between that weight and the output. This is literally the chain rule applied in sequence.

*Footnote: d/dx[f(g(x))] = f'(g(x)) * g'(x). Product rule: d/dx[f*g] = f'g + fg'. Quotient rule: d/dx[f/g] = (f'g - fg') / g^2.*

Chapter 5

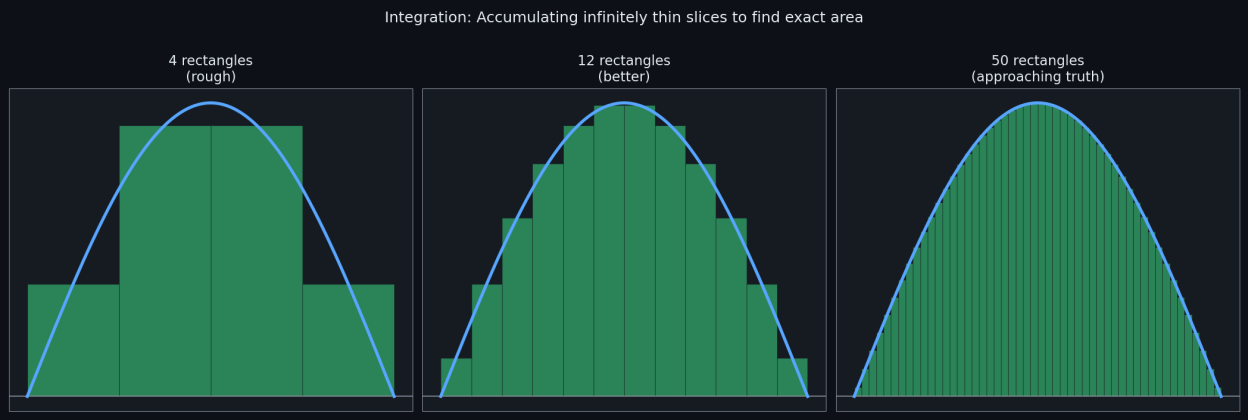
The Integral

If the derivative is about instantaneous rates of change, the integral is about *accumulation*. How much has accumulated? How much area is under this curve? How much distance has been covered given a velocity at every moment? Integration answers all of these.

Slices of Infinite Thinness

Imagine measuring the area under a curve by filling it with rectangles. Use fat rectangles — the estimate is rough. Use thinner ones — better. Keep going: thinner and thinner, more and more of them. In the limit of infinitely thin rectangles, you get the exact area. This process is integration.

The magic is that 'sum of infinitely many infinitely thin things' gives you a perfectly finite, exact answer. This is one of the beautiful counterintuitive truths of calculus.



Riemann sums converging to the true area under sin(x) from 0 to pi. 4 rectangles: rough. 12: better. 50: nearly exact. Infinite: perfect.

What the Integral Actually Measures

The integral of a velocity function gives you position. The integral of a power function gives you energy. The integral of a probability density gives you probability. The integral of a marginal cost gives you total cost. In every case, integration is *adding up continuously varying quantities*.

The definite integral gives you a single number — the total accumulation between two bounds. The indefinite integral gives you a function — the running total up to any point.

ML / AI Connection

In probability theory, every probability distribution has a PDF (probability density). The probability that a value falls in a range is the integral of the PDF over that range. Loss functions like cross-entropy are defined using logarithms and integrals of distributions. KL divergence — the core of VAEs and information theory in ML — is an integral.

*Footnote: integral from a to b of f(x)dx = lim_{n→inf} sum_{i=1}^{n} f(x_i) * delta_x where delta_x = (b-a)/n. The symbol integral sign is an elongated 'S' for 'sum'.*

Chapter 6

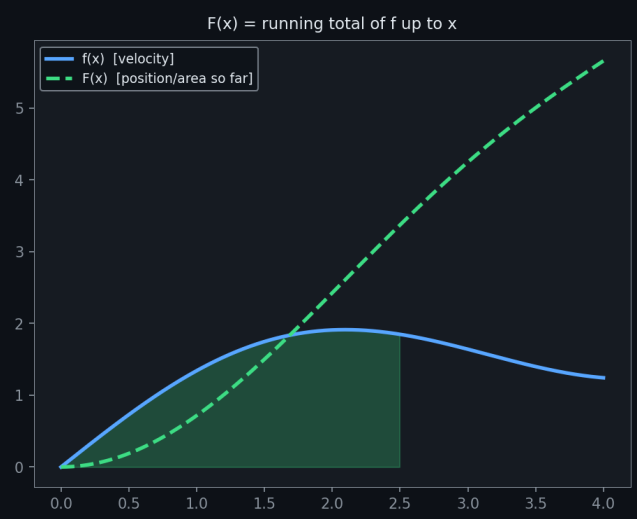
The Fundamental Theorem of Calculus

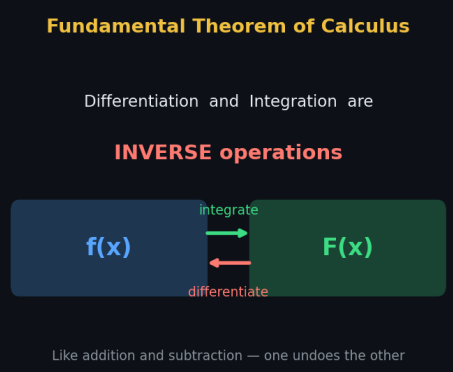
This is the central revelation of calculus: differentiation and integration are not two separate ideas. They are inverse operations of each other. Like multiplication and division. Like addition and subtraction. One undoes the other.

The Two-Way Bridge

If you integrate a function and then differentiate the result, you get back to where you started. If you differentiate a function and then integrate, same thing. The derivative tells you the slope. Integration adds up slopes to recover the height. They undo each other perfectly.

Why is this remarkable? Because when calculus was first being developed, the area problem and the slope problem seemed completely unrelated. Newton and Leibniz discovered they were the same problem — two faces of one coin.





Left: F(x) is the running area under f. The slope of F at any x equals f(x). Right: the bidirectional relationship — integration and differentiation cancel out.

The Practical Meaning

The theorem tells you: to find the exact area under a curve from a to b, you only need to find a function whose derivative is the curve (called the antiderivative). Then evaluate it at both ends and subtract. No rectangles. No limits. Just one elegant calculation.

This turns integration from an impossible infinite sum into something you can do by recognizing patterns — working backwards from derivatives you already know.

ML / AI Connection

In physics simulations used for robot learning and game AI, the FTC appears constantly: integrating accelerations gives velocities, integrating velocities gives positions. Energy calculations in physics-based ML environments are integrals over time.

Footnote: FTC Part 1: d/dx[integral from a to x of f(t)dt] = f(x). FTC Part 2: integral from a to b of f(x)dx = F(b) - F(a) where F'=f.

Chapter 7

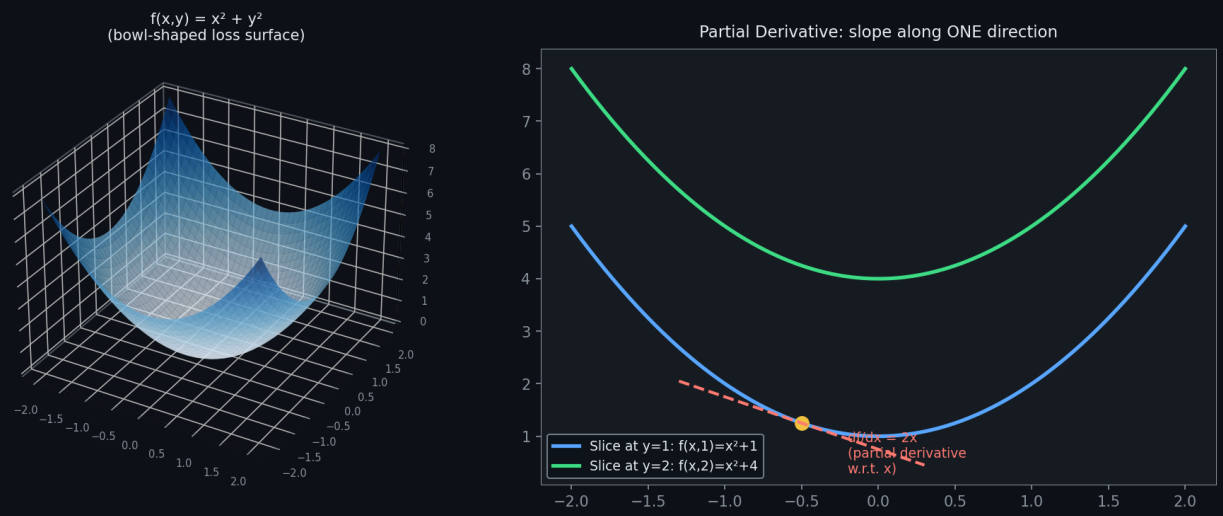
Multivariable Calculus

Real ML models have not one but millions of inputs — the weights. Multivariable calculus extends all the ideas of derivatives to functions with many inputs. Instead of a slope on a curve, you get a slope landscape: a surface that can slope differently in every direction.

Partial Derivatives — Slicing a Mountain

If a function takes two inputs — say x and y — you can ask: 'if I hold y fixed and only nudge x, how does the output change?' That is the partial derivative with respect to x. You are slicing the 3D surface with a vertical plane and measuring the slope of that slice.

Do the same for y. Now you have two slopes, one for each direction. This generalises to any number of inputs. A function with 1 million inputs has 1 million partial derivatives.



Left: f(x,y) = x² + y² is a bowl-shaped surface. Each horizontal slice is a curve, and the slope of that curve is a partial derivative. Right: the partial derivative df/dx seen in a 2D slice.

The Gradient — The Direction of Steepest Climb

Collect all the partial derivatives into a single vector: that is the gradient. The gradient has a beautiful geometric meaning: it points in the direction of steepest ascent on the surface. Like a hiker on a hill, the gradient is the compass direction that takes you uphill fastest.

The negative gradient therefore points downhill. And downhill is where lower loss lives. This is the entire reason gradient descent works.

ML / AI Connection

A neural network with 175 billion parameters (like GPT-3) has a gradient vector of 175 billion numbers. Each number says: 'nudge this weight in this direction ' by this much to reduce the loss.' The optimizer uses this gradient at every training step.

Footnote: gradient of f = nabla f = (df/dx_1, df/dx_2, ..., df/dx_n). The gradient at a point is perpendicular to the level curves (contour lines) of the function.

Chapter 8

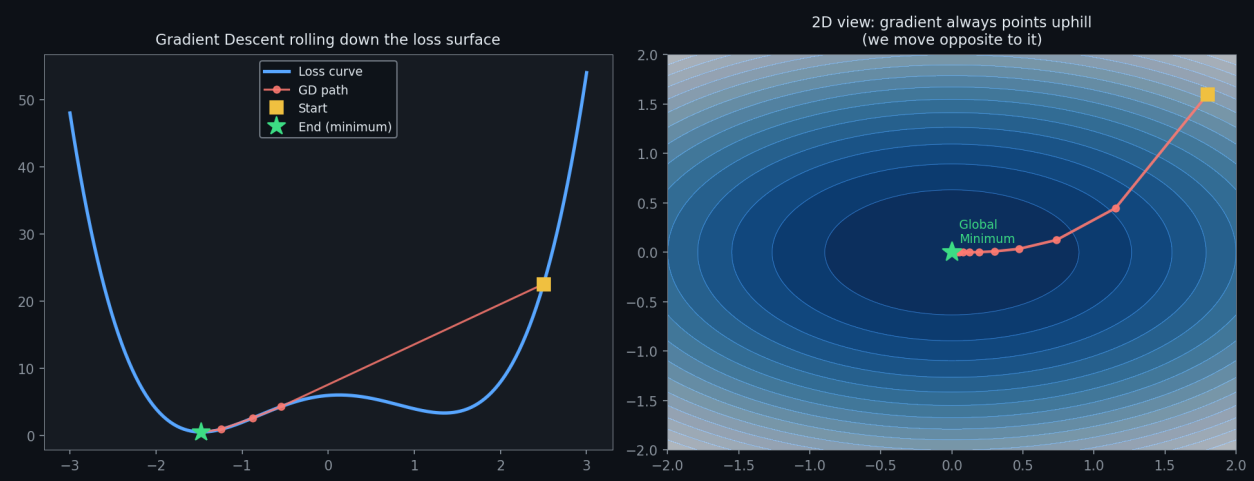
Gradient Descent ★

Gradient descent is the algorithm that trains nearly every modern ML model. It is built entirely on the idea of the gradient: follow the direction of steepest descent, one small step at a time, until you reach the bottom of the loss surface.

Rolling Down a Hill

Imagine you are blindfolded on a hilly terrain and want to reach the lowest valley. You cannot see. But you can feel the ground under your feet — which direction is it sloping? You take one step downhill. Feel again. Take another step. Repeat. Eventually you reach a valley.

This is gradient descent. The terrain is the loss surface — a landscape where height represents error. The blindfold represents the fact that you cannot see the whole landscape at once. Each step uses the gradient to determine direction.



Left: gradient descent path on a 1D loss curve. It starts far from the minimum (yellow square) and converges (green star) by repeatedly stepping opposite to the gradient. Right: the same idea in 2D, shown as contour lines.

Learning Rate — The Step Size

How large should each step be? This is the learning rate. Too large: you overshoot the valley, bouncing wildly. Too small: training takes forever. The learning rate is one of the most important hyperparameters in training any model.

Visually: a large learning rate means leaping. A small one means shuffling. The ideal is somewhere between — big enough to make progress, small enough to not fly past the minimum.

Variants: SGD, Adam, and Beyond

Pure gradient descent computes the gradient over all training data — which is expensive. Stochastic Gradient Descent (SGD) uses a random mini-batch at each step: noisier, but dramatically faster. Adam adds momentum and adaptive per-parameter step sizes, making it the default optimizer for most

modern networks.

ML / AI Connection

Every time you see a training loss curve decreasing, you are watching gradient descent work. The bump and noise in the curve? That is SGD sampling different mini-batches. When the curve flattens, the gradient is near zero — you are near a minimum. Adam, AdaGrad, RMSProp — all are smarter versions of this same core idea.

*Footnote: w = w - lr * gradient(L). Adam: m = beta1*m + (1-beta1)*g; v = beta2*v + (1-beta2)*g^2; w = w - lr * m / (sqrt(v) + epsilon). lr = learning rate, g = gradient, m = 1st moment, v = 2nd moment.*

Chapter 9

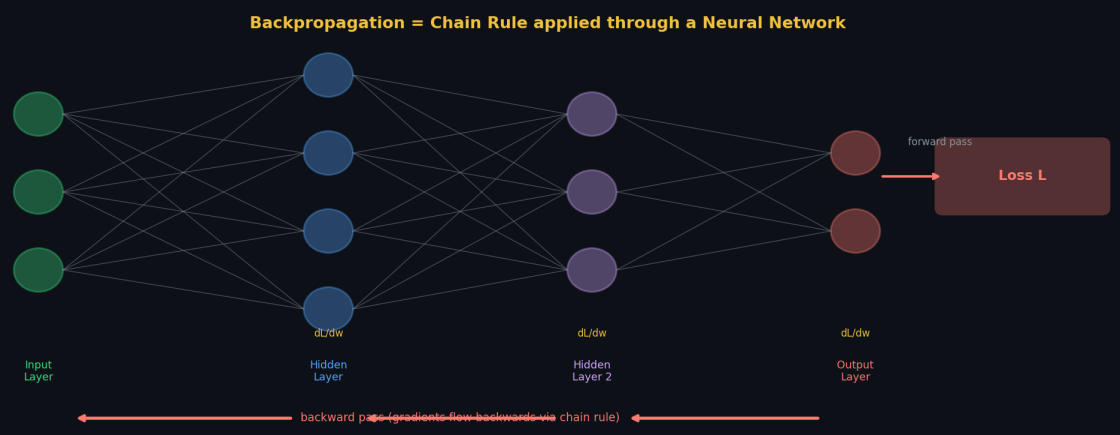
Backpropagation ★

Backpropagation is how neural networks learn. It is the chain rule applied systematically from the output back to the input, computing how each weight contributed to the final error. Without this algorithm, modern deep learning would not exist.

The Forward and Backward Pass

Every training step has two phases. First, the forward pass: data flows from input, through every layer, to a prediction. The loss is computed — how wrong were we?

Then the backward pass: the error flows backwards. Using the chain rule, we calculate how each weight in each layer contributed to that error. This gives us the gradient for every single weight simultaneously.



Forward pass (left to right): data flows through layers to produce a prediction and a loss. Backward pass (right to left): gradients flow back through each layer via the chain rule, telling each weight how to update.

Why the Chain Rule Is the Key

The final loss depends on the output layer. The output layer depends on the previous layer's activations. Those depend on the layer before. And so on, all the way to the first weights. The chain rule lets you compose these dependencies: 'how does a tiny change in weight W_1 affect the final loss?' — multiply the derivative through every layer in between.

This would be impossibly expensive if done naively. Backprop is efficient because it reuses intermediate computations (the activations stored during the forward pass) to avoid recalculating them. This is the key engineering insight that makes modern deep learning fast enough to run.

ML / AI Connection

Automatic differentiation (autograd) in PyTorch and TensorFlow implements this exactly: it builds a computation graph during the forward pass, then traverses that graph in reverse during the backward pass, applying the chain rule at each node. Every .backward() call in PyTorch is backpropagation via the chain rule.

*Footnote: for a network L = f_n(f_{n-1}(...f_1(x))), dL/dw_1 = (dL/da_n) * (da_n/da_{n-1}) * ... * (da_2/dw_1) where a_i are the layer activations. Each factor is a Jacobian matrix.*

Chapter 10

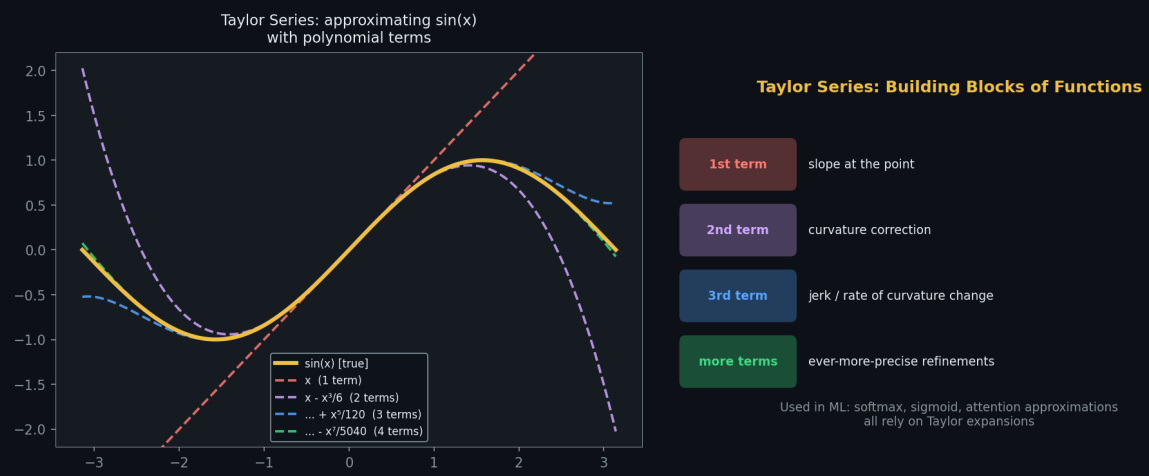
Taylor Series

A Taylor series is a way of expressing any smooth function as an infinite sum of polynomial terms. Each term captures more and more detail about the function's shape. The result: any complex function can be approximated — often very accurately — by simple polynomials.

The Idea: Building a Function From Its Derivatives

If you know a function's value at a point, and its slope there, and how the slope is changing, and how fast that change is changing... you can reconstruct the entire function. Each piece of information you add refines the approximation. The Taylor series systematically encodes all of this into polynomial terms.

The first term: where you are (constant). The second term: which direction the curve goes (slope = first derivative). The third term: how the slope itself curves (curvature = second derivative). And so on, forever.



Left: Taylor approximations to sin(x). Each additional term extends accurate agreement further from the expansion point. Right: what each term geometrically captures.

Applications in ML

Taylor series appear constantly in ML — often invisibly. The sigmoid function's gentle S-curve is understood through its Taylor expansion. The softmax function is analyzed using Taylor approximations of the exponential. Second-order optimizers like Newton's method use the quadratic (two-term) Taylor approximation of the loss surface to take much larger, smarter steps than gradient descent.

ML / AI Connection

GELU activation (used in GPT models) was derived by analyzing the Taylor expansion of the Gaussian CDF. Attention mechanisms use the Taylor approximation of softmax for linear attention variants. Second-order optimizers like L-BFGS approximate the Hessian to use the Taylor expansion of the loss landscape for more efficient convergence.

Footnote: $f(x) = f(a) + f'(a)(x-a) + f''(a)(x-a)^2/2! + f'''(a)(x-a)^3/3! + \ldots$ Maclaurin series: expansion around $a=0$. $e^x = 1 + x + x^2/2! + x^3/3! + \ldots$ (fundamental to all ML involving exponentials).

Chapter 11

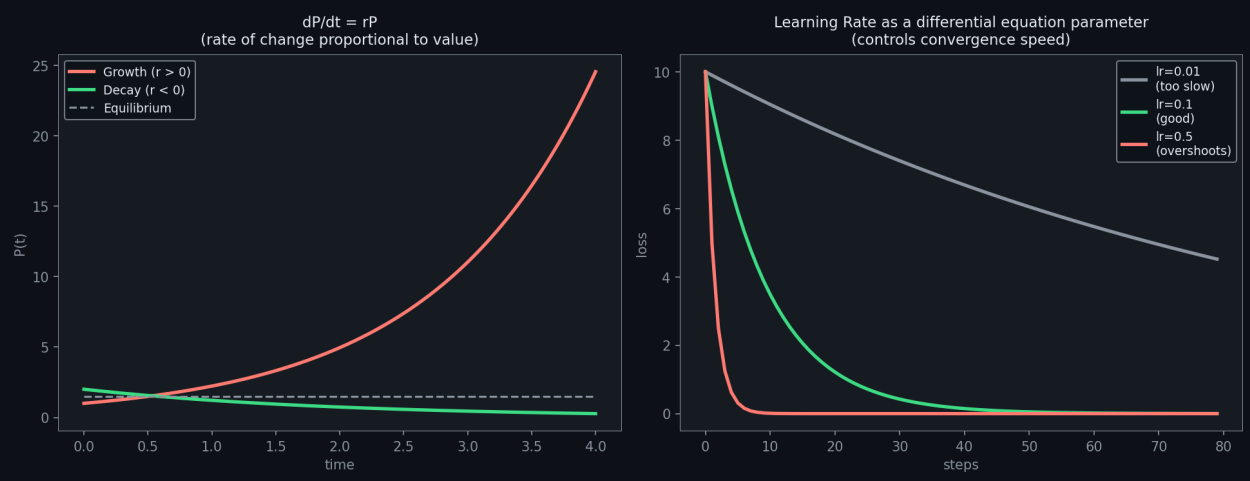
Differential Equations

A differential equation describes how something *changes*, not what it *is*. Instead of saying 'the population is 1000,' it says 'the population grows at a rate proportional to itself.' You solve it to find what the population actually is over time.

The Core Idea: Change Described by Its Own Rate

Almost everything in the natural world is described by differential equations: how heat spreads, how populations grow, how electrical signals propagate through neurons. A differential equation is a relationship between a quantity and its own rate of change.

The simplest case: if something grows proportionally to how much there is, you get exponential growth. This single differential equation — the simplest possible — explains compound interest, viral spread, radioactive decay, and the early stages of neural network weight growth.



Left: dP/dt = rP — exponential growth or decay depending on r. Right: the same concept applied to neural network training — the learning rate is the parameter r that controls convergence behavior.

Neural ODEs — Differential Equations as Layers

A remarkable recent discovery: a neural network with infinitely many layers, each making an infinitely small change, is equivalent to a differential equation. This led to Neural ODEs — a class of models where the network architecture is defined by a differential equation rather than discrete layers.

Instead of saying 'compute layer 1, then layer 2, then layer 3,' you say 'let the hidden state evolve continuously from time 0 to time 1.' This enables models with continuous depth, better memory efficiency, and elegant handling of time-series data.

ML / AI Connection

Neural ODEs are used in time-series forecasting, physics simulations, and generative models. Diffusion models — behind DALL-E, Stable Diffusion, and Sora — are defined by a stochastic differential equation (SDE): they gradually add noise according to a noise schedule, then learn to reverse the SDE to generate images from noise.

Footnote: ODE = Ordinary Differential Equation. SDE = Stochastic Differential Equation (includes a noise term). Euler's method solves ODEs numerically: $y_{t+1} = y_t + dt * f(y_t, t)$.

Chapter 12

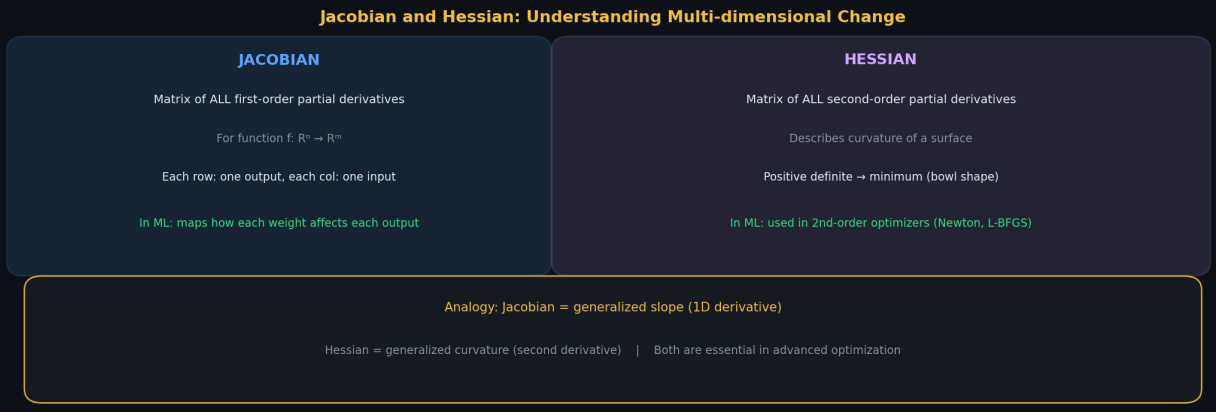
Jacobian and Hessian ★

When you have many inputs and many outputs, a single number cannot describe how one changes relative to the other. You need a matrix. The Jacobian is that matrix for first derivatives. The Hessian does the same for second derivatives.

The Jacobian: All First Derivatives at Once

For a function that takes n inputs and produces m outputs, the Jacobian is an m-by-n matrix. Each entry tells you: 'how does output i change when input j is nudged?' The Jacobian is the complete first-order picture of how a transformation works.

Geometrically, the Jacobian at a point describes how a tiny region near that point gets stretched, rotated, or sheared by the function. The determinant of the Jacobian tells you by how much area (or volume) scales.



The Jacobian and Hessian compared by purpose, structure, and ML application. The Jacobian generalizes the first derivative; the Hessian generalizes the second.

The Hessian: Curvature in Every Direction

The Hessian matrix collects all second-order partial derivatives. Where the gradient tells you which way the surface slopes, the Hessian tells you how the slope itself is changing in every direction. This is information about curvature: is the surface a gentle slope or a sharp cliff? Is this a true minimum or a saddle point?

A positive definite Hessian at a point means the surface is bowl-shaped there — a genuine local minimum. A negative definite Hessian means a local maximum. An indefinite Hessian means a saddle point — downhill in some directions, uphill in others.

ML / AI Connection

Saddle points are extremely common in high-dimensional loss surfaces. A network with millions of parameters is almost never at a true local minimum — it is usually at a saddle point. First-order optimizers (gradient descent) struggle near saddle points; second-order methods (Newton, L-BFGS) use the Hessian to navigate them more cleverly. The Hessian also appears in Fisher information matrices, natural gradient methods, and Bayesian deep learning.

*Footnote: Jacobian J[i,j] = df_i/dx_j. Hessian H[i,j] = d^2f/(dx_i * dx_j). The Hessian is always symmetric (Schwarz's theorem). Eigenvalues of the Hessian reveal the curvature in each principal direction.*

Chapter 13

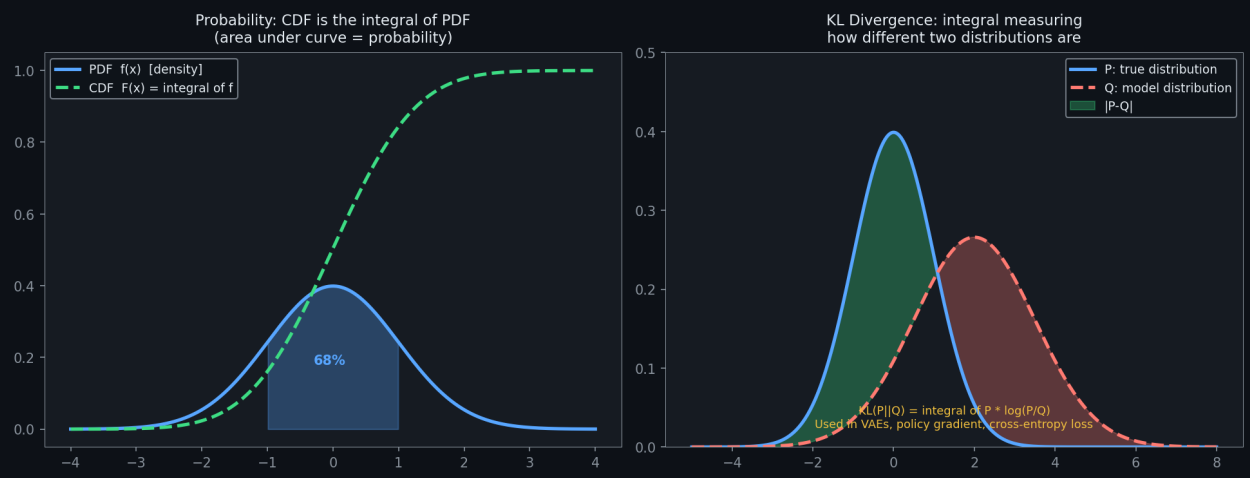
Calculus and Probability ★

Probability and calculus are inseparable when dealing with continuous variables. A probability distribution over continuous values is described by a density function, and probabilities are integrals of that density. Loss functions, uncertainty measures, and training objectives in ML are built on this foundation.

Probability Density: Area Is Probability

For continuous values (like the height of a person, or a neural network's output before it is quantized), you cannot assign a probability to an exact value — the probability of being exactly 1.7324... metres tall is zero. Instead, you assign probability to ranges.

The probability density function (PDF) describes how probability is spread. The probability that a value falls between a and b is the integral of the PDF from a to b. Taller density = more probability packed into that region.



Left: the PDF (blue) and CDF (green) of a normal distribution. The shaded area (68%) is the integral of the PDF between -1 and +1 sigma. Right: KL divergence as the integral of the signed difference between two distributions.

KL Divergence and Cross-Entropy — Integral Loss Functions

KL (Kullback-Leibler) divergence measures how different one distribution is from another. It is defined as an integral. Intuitively: it measures how surprised you would be if you expected one distribution but observed samples from another.

Cross-entropy loss — the loss function used to train most classifiers — is deeply connected to KL divergence. Minimizing cross-entropy loss is equivalent to minimizing the KL divergence between the model's predicted distribution and the true data distribution.

ML / AI Connection

Every time a model trains with cross-entropy loss, it is minimizing an integral-based measure of distributional mismatch. Variational Autoencoders (VAEs) explicitly minimize KL divergence as part of their loss. Diffusion models are trained by matching score functions (derivatives of log probability densities). Information theory — the mathematical backbone of compression and generalization — is entirely built from integrals over probability distributions.

*Footnote: KL(P||Q) = integral of P(x) * log(P(x)/Q(x)) dx. Cross-entropy H(P,Q) = -integral of P(x) * log(Q(x)) dx = H(P) + KL(P||Q). Entropy H(P) = -integral of P(x) * log(P(x)) dx.*

Chapter 14

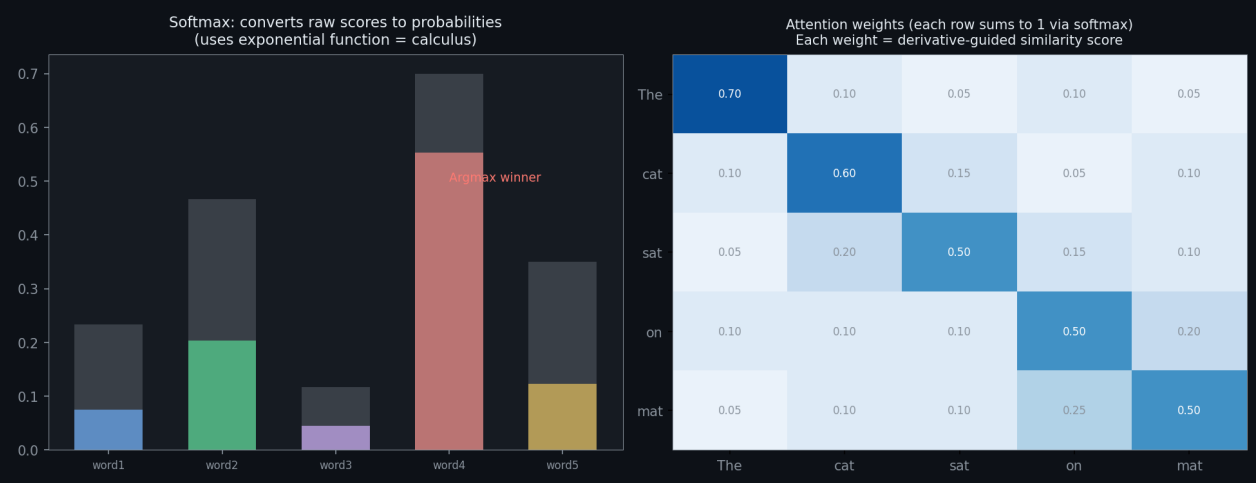
Attention and Softmax ★

The Transformer — the architecture behind GPT, BERT, and virtually every modern large language model — is built on a mechanism called attention. At its heart, attention is a calculus operation: a weighted sum where the weights are computed using derivatives.

Softmax: From Scores to Probabilities via Calculus

Softmax converts a vector of raw scores (called logits) into a probability distribution. It does this by exponentiating each score — which is a calculus operation — and then normalizing. The exponentiation amplifies differences: the highest score gets a much larger weight than others.

Why use the exponential? Because its derivative is itself. This beautiful self-referential property makes softmax exceptionally well-behaved under differentiation — gradients flow cleanly through it during backpropagation.



Left: softmax converting raw logits to probabilities. The highest logit (word4) dominates, but all words get nonzero weight. Right: an attention matrix showing how each token attends to others. Each row sums to 1 (via softmax).

Attention: A Differentiable Memory Lookup

Attention is like a search engine inside the model. A query asks 'what am I looking for?' Keys describe 'what do I have available?' Values are 'what do I actually give you if you select me?' The dot product between a query and each key produces similarity scores. Softmax converts these to weights. The output is a weighted sum of values.

The entire process is differentiable. Every step can have a gradient computed. This is what makes it trainable: the model learns, via gradient descent, which queries to make and which keys to listen to, purely from the training signal.

ML / AI Connection

Self-attention computes $O(n^2)$ interactions between n tokens — that is why Transformers struggle with very long contexts. Linear attention approximates the softmax using Taylor expansions of the exponential, reducing this to $O(n)$. Flash attention rearranges the integral-like computation to fit within GPU memory hierarchies. Every optimization of the attention mechanism is a calculus optimization.

Footnote: $Attention(Q,K,V) = softmax(QK^T / \sqrt{d_k}) * V$. Softmax: $\sigma(z)_i = e^{z_i} / \sum_j(e^{z_j})$. $d/dz_i \, softmax_i = softmax_i * (1 - softmax_i)$ (elegant self-referential gradient).

Bringing It All Together

Calculus is not a collection of tricks for solving exam problems. It is the language in which the universe describes change. And machine learning — at its core — is the science of measuring change, following change, and using change to improve.

Every weight update in a neural network: a derivative. Every training step: gradient descent following derivatives downhill. Every loss function: an expression to be minimized via calculus. Every attention weight: a softmax — an exponential with beautiful derivative properties. Every probability in a distribution: an integral.

Concept	Visual Meaning	ML / AI Use
Function	Machine: input → output	Neural network layers, activations
Limit	Value approached, not reached	Continuity, subgradients of ReLU
Derivative	Slope at one point	Gradient, weight update direction
Chain Rule	Rates multiply through pipelines	Backpropagation through layers
Integral	Accumulated area under curve	Probability, KL divergence, cross-entropy
FTC	Integration undoes differentiation	Physics engines, energy in RL
Gradient	Vector of all partial derivatives	The update direction for all weights
Gradient Desc.	Roll downhill on loss surface	The engine of every optimizer
Backprop	Chain rule backwards through network	How every neural net trains
Taylor Series	Polynomial approximation to any f	GELU, linear attention, 2nd-order opt.
Jacobian	Matrix of first derivatives	Layer-wise sensitivity, autograd
Hessian	Matrix of second derivatives	Curvature, saddle points, L-BFGS
PDF / Integral	Area = probability	Loss functions, VAE, diffusion models
Softmax	Exponential → probabilities	Attention, classification heads

The goal was never to make you able to differentiate x^3 faster. The goal is to make you *see*: a derivative is a zoom, an integral is a slice, a gradient is a compass, and training is just rolling a ball downhill on a surface you shaped yourself.

Now when you see a loss curve, you see derivatives. When you see an optimizer, you see gradient descent. When you see attention, you see softmax and calculus working together. The maths was always visual — it just needed the right frame.

Further Visual Learning

- 3Blue1Brown — Essence of Calculus: youtube.com/playlist — the direct visual inspiration for this document
- 3Blue1Brown — Neural Networks: youtube.com — visualizing backprop and gradient descent

- Distill.pub — interactive research articles with visual explanations of ML concepts
- Khan Academy Calculus — for building calculation fluency once the intuition is solid